

Agentic Research Pipeline

Repository, architecture and process-flow map · generated with graphify (AST) · 72,013 backend lines

712
code files

8,230
graph nodes

29,219
edges

207
communities

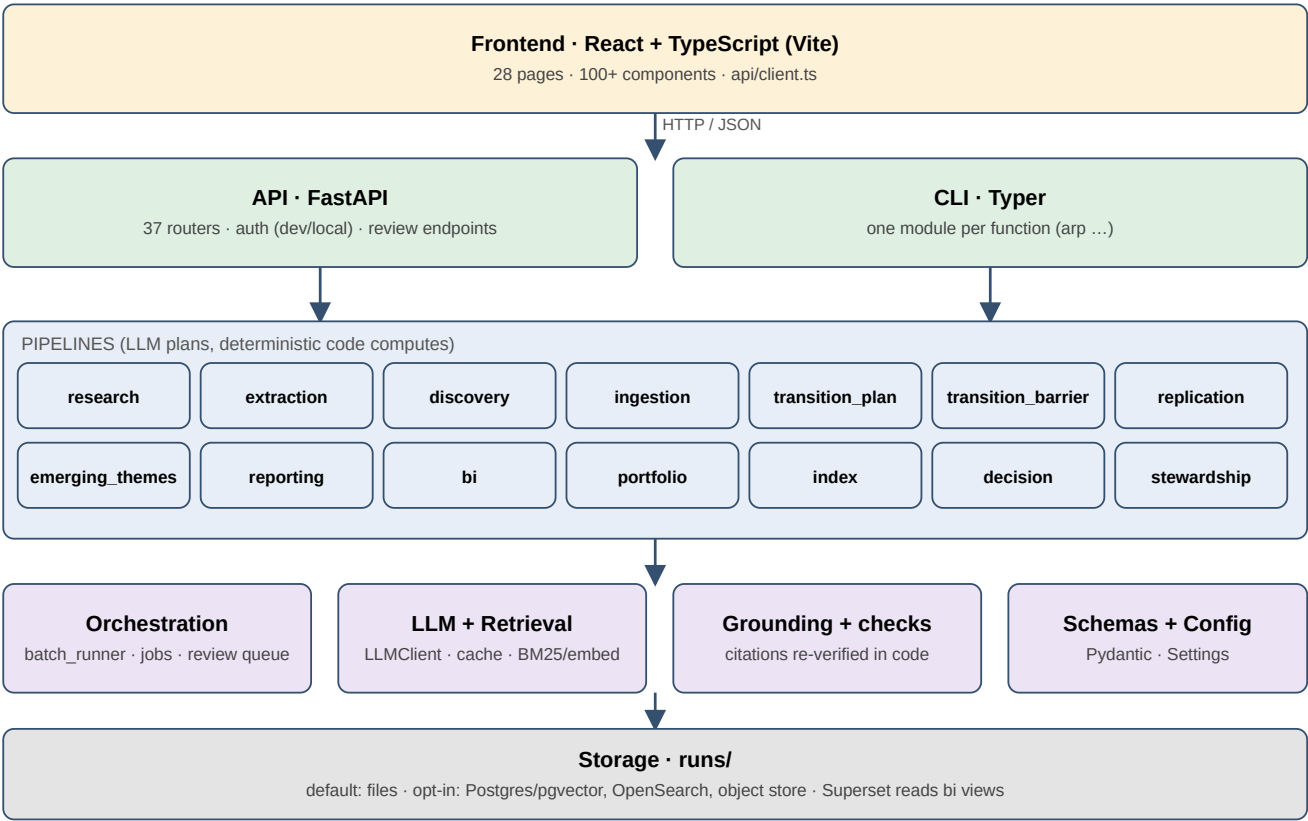
What it is

Agent pipelines for investment research at scale (up to ~4,000 companies per run): thematic universes, schema-defined data-point extraction, portfolio and climate analytics, index construction and decision scoring. Python backend (FastAPI + Typer CLI), React/TypeScript frontend, file-based run state by default.

Two design rules that shape the code

- **The LLM plans, deterministic code computes.** Decision Studio, indices, exposures and backtests use zero LLM calls for numbers.
- **Every citation is re-verified programmatically** (arp/grounding.py) against the source text, never trusted from the model.

Layered architecture



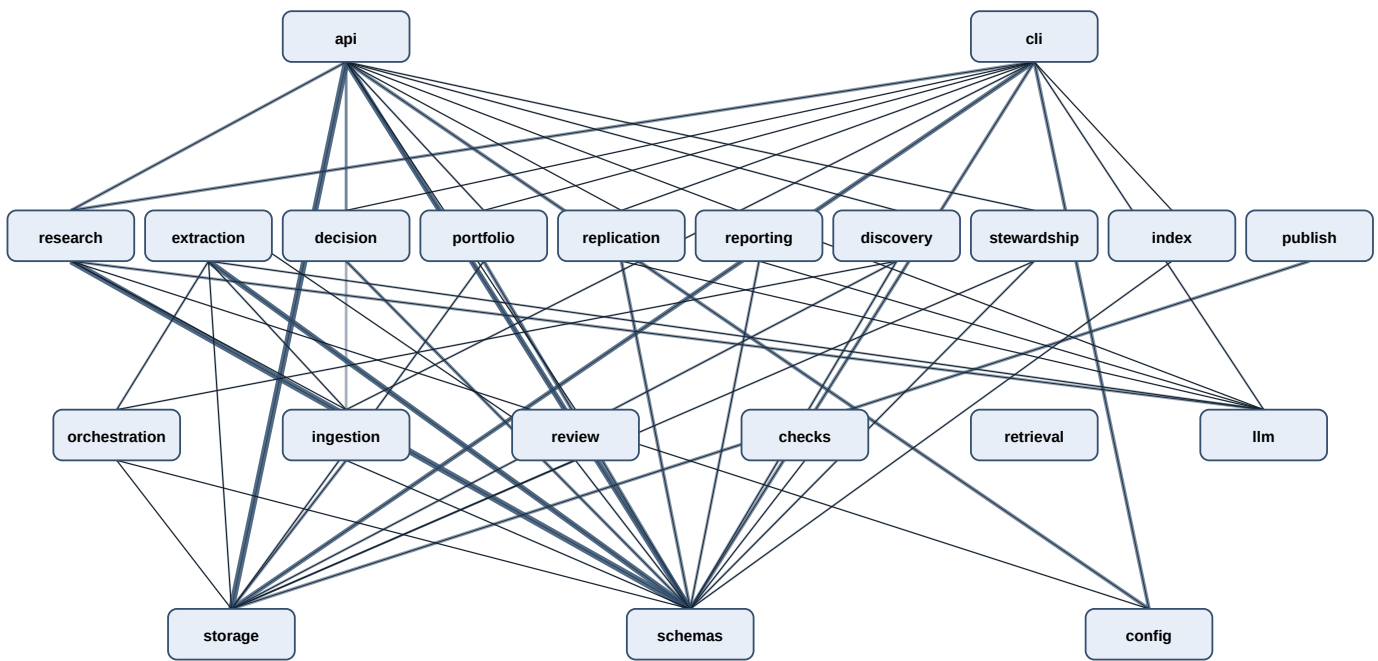
1 · Package map

Files, lines, and fan-in (how many other packages import it). Computed from import statements in backend/arp.

Package	Files	Lines	Fan-in	Role
api	43	8,092	7	FastAPI app, 37 routers, auth, review endpoints
storage	36	6,076	30	File/Postgres/OpenSearch/object stores, run store
schemas	36	5,438	33	Pydantic models shared by everything
cli	33	4,799	0	Typer CLI, one module per function
stewardship	19	4,294	2	Proxy voting, engagement, steward workflows
decision	23	4,151	7	Decision Studio scoring engine (zero LLM)
extraction	27	3,907	12	Data-point, financials, TNFD extract/verify graphs
research	42	3,903	6	Universe builder: match graph, debate, exposures
index	13	3,096	3	Equity index construction, capping, PAB/CTB
reporting	24	3,093	3	pptx/docx/pdf renderers, house decks
portfolio	26	2,630	7	Holdings, climate analytics, NL Q&A, monitoring
replication	24	2,621	2	Strategy spec + deterministic backtests
ingestion	13	2,351	18	Document intake, EDGAR, ESEF, XBRL, parsing
emerging_themes	17	1,676	4	Bottom-up theme scanner
bi	12	1,666	5	Superset BI designer (plan, validate, compile)
discovery	13	1,593	8	IR-site crawl, downloads, identity resolution
orchestration	9	1,291	18	Batch runner, job manager, review queue, cost
review	6	1,078	4	Review decisions, analytics, quality
publish	7	1,062	5	Reviewed-facts release gate
transition_plan	10	843	5	64-indicator walk-vs-talk assessment
checks	9	691	4	Consistency, plausibility, cross-source checks
voting	6	665	3	Proxy voting runs
holdings	6	631	7	Holdings aggregation
transition_barrier	9	623	3	105-cell sector barrier matrix
engagement	7	595	4	Engagement tracking
normalise	6	583	5	Units and typed values
config	1	579	24	Settings (env-driven)
llm	5	480	18	LLMClient interface, LangChain client, cache
retrieval	6	478	14	BM25 + embedding evidence selection
presets	5	460	2	Preset configs
snapshots	4	451	4	Point-in-time snapshots
projects	4	435	3	Project containers
agents	3	340	2	Taxonomy Researcher, Calibration Agent
grounding	1	315	10	Citation re-verification vs source text
golden_set	5	280	2	Evaluation sets
planning	5	227	2	Field applicability, period and doc routing
scale	3	165	1	Scale helpers
retention	1	147	1	Retention policy
catalog	1	114	1	Catalog
net_safety	1	56	4	SSRF / network safety
universe	1	33	6	
__init__	1	5	0	

2 · Package dependencies

Top 55 cross-package import edges, drawn downward; thicker means more imports. Everything funnels into storage, schemas and config at the bottom; api and cli sit on top.



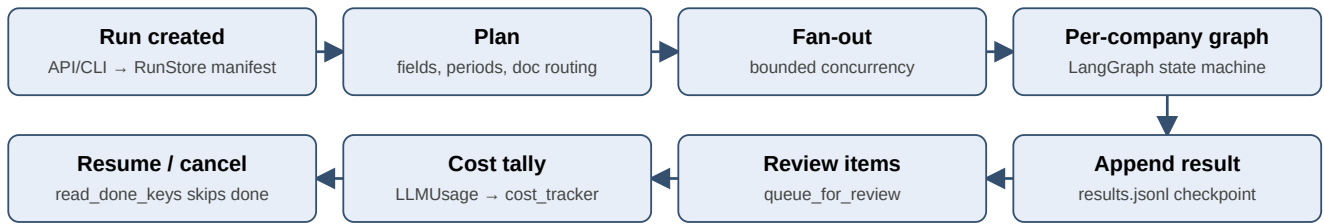
Hubs in the code graph (god nodes)

Node	Degree	Why it matters
RunStore	386	every pipeline reads/writes run state through it
Settings	320	env-driven config threaded everywhere
LLMClient	251	single narrow LLM interface
CompanyRef	244	canonical company identity
LLMUsage	216	token/cost accounting
PortfolioStore	177	holdings persistence
Principal	134	authenticated user for audit trails
EngagementStore	121	engagement persistence

Reading: **RunStore** is the single most connected symbol. Changing its interface touches nearly every pipeline. **LLMClient** keeps the model provider swappable behind one interface.

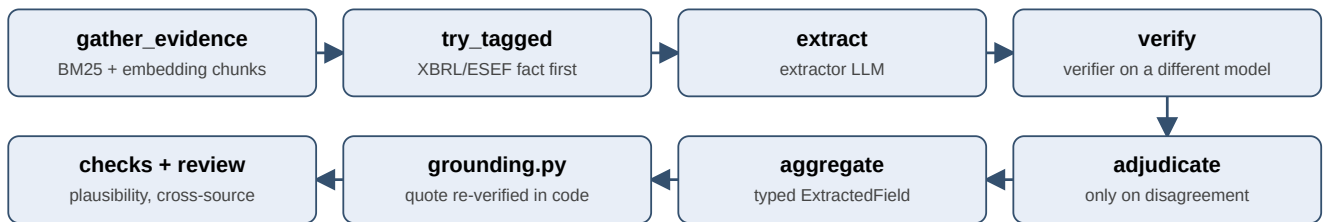
3 · Process flows

A · Any per-company run (shared by every pipeline)



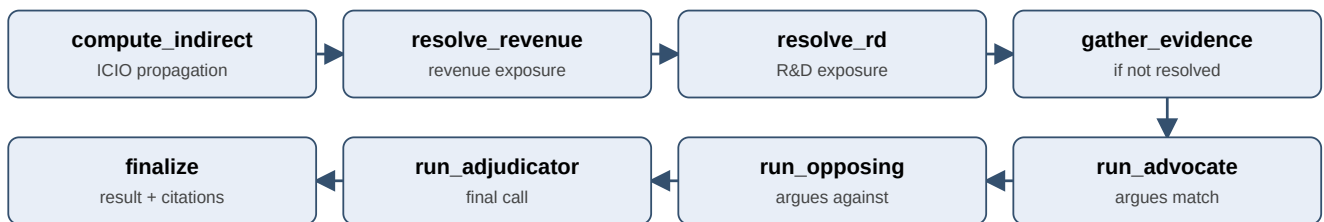
orchestration/batch_runner.py: results append to runs/<id>/results.jsonl as each company finishes; items a worker stops for review are recorded and never redone on resume.

B · Data-point extraction (extraction/graph_shape.py)



An independent verifier on a different model; the adjudicator runs only when they disagree. Structured XBRL/ESEF facts are tried first where available.

C · Thematic universe match (research/match_graph.py)



Quantitative exposure resolves first; the Advocate / Opposing / Adjudicator debate runs only when evidence is needed.

4 · Surfaces and storage

Frontend (frontend/src)

28 pages (ThemeBuilder, Extraction, DecisionStudio, IndexBuilder, ReviewQueue, PortfolioRiskMonitoringTool, ReportBuilder, StrategyReplication, TransitionBarrierAssessment, VotingRuns ...), shared components, `api/client.ts` as the only backend gateway, `lib/` helpers (runs, jobs, review keys, rule graphs, zen decision engine worker).

API and CLI

37 routers under `api/routers` map nearly 1:1 to CLI modules under `cli/` (taxonomy, extraction, universe, index, decision, portfolio, ...). Auth: dev mode (loopback signs in as approver) or local token mode with viewer / analyst / approver roles; four-eyes steps need local mode.

Storage

Default is file-based: `runs/`, `taxonomies/`, `portfolios/`, `frameworks/`, `indices/`. Opt-in backends: Postgres + pgvector, OpenSearch, object store (docker-compose). Superset reads only the `bi` views over Postgres.

Reviewed-facts path

extraction results → review queue → review decisions (audit trail per field) → `publish/` gate → reviewed facts consumed by BI and downstream functions.

Caveats

- Graph built from AST of backend/arp and frontend/src only (tests, docs, mocks excluded); docs-derived semantic edges were not extracted.
- Package dependency counts are import statements, not runtime call counts. Relative imports are not counted.